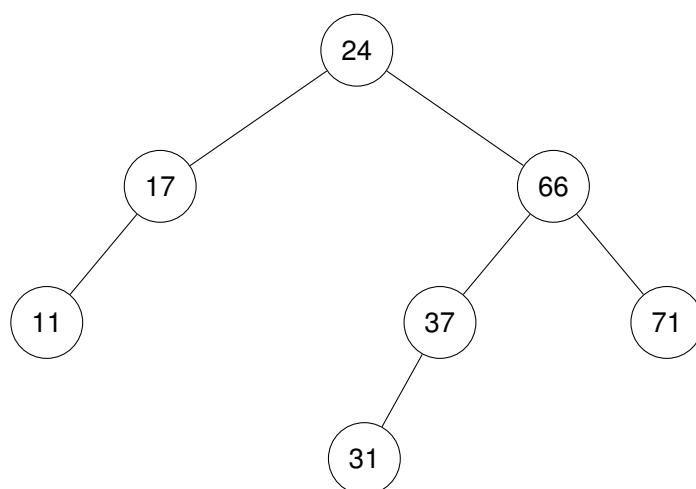
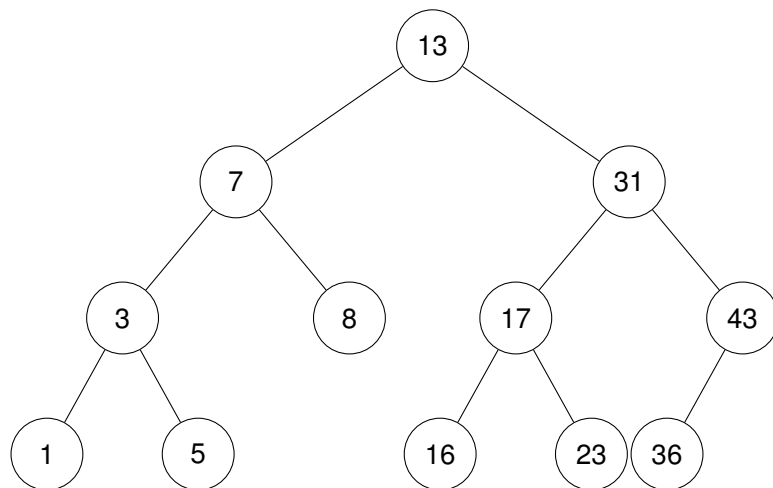


Gabarito — Lista de exercícios — Árvores AVL

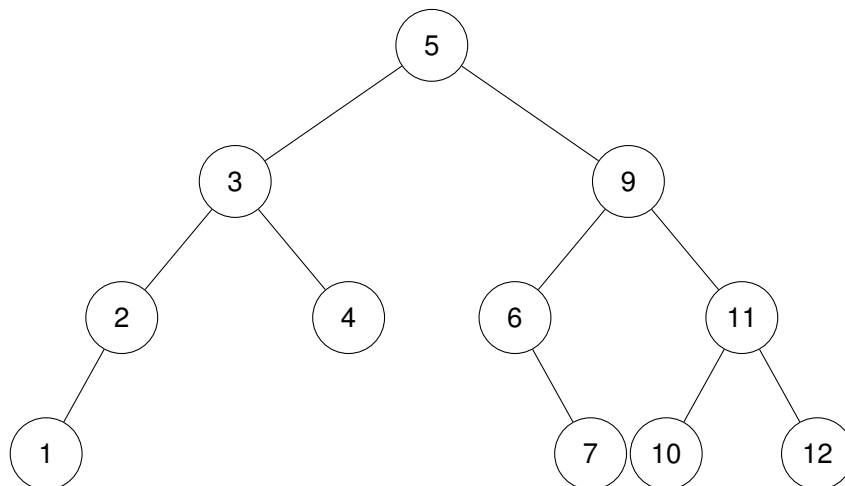
1. (d).
2. (c).
3. V, V, F, V, V.
4. (c).
5. Árvore final:



6. Árvore final:



7. Após remover 8, a árvore resultante é:

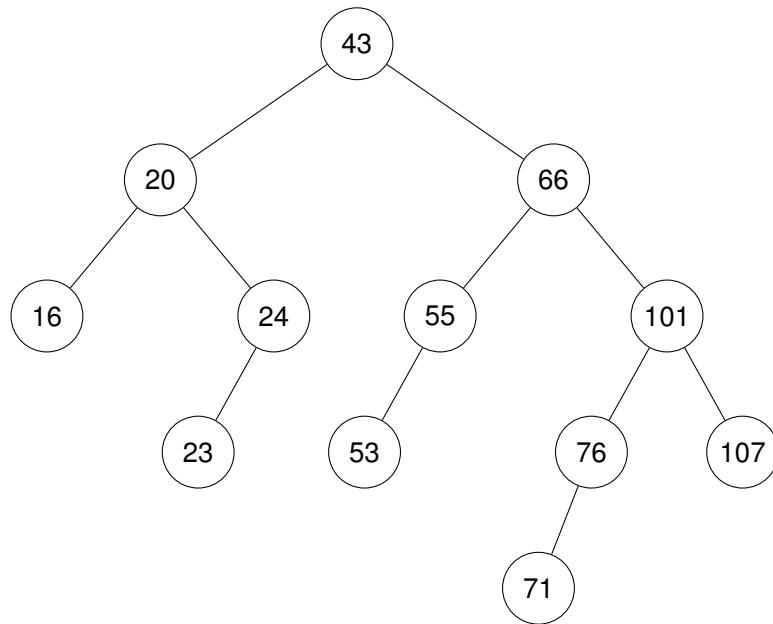


A inserção de 9 não altera a árvore, pois o valor já está presente.

8. Resultados sequenciais:

- (a) após inserir 107: nenhuma rotação;
- (b) após inserir 20: rotação dupla esquerda-direita na subárvore de 24;
- (c) após remover 99: nenhuma rotação;
- (d) após remover 36: rotação simples à direita na subárvore esquerda.

Árvore final:



9. Em todos os casos, a árvore resultante permanece AVL **sem necessidade de rotação**.

- (a) inserir 70: nenhuma rotação;
- (b) inserir 45: nenhuma rotação;
- (c) remover 10: nenhuma rotação;
- (d) remover 60: nenhuma rotação.

10. (c).

```

11. int contarNosComDoisFilhos(NoArv* raiz) {
    if (raiz == NULL) return 0;
    int atual = (raiz->esq != NULL && raiz->dir != NULL) ? 1 : 0;
    return atual + contarNosComDoisFilhos(raiz->esq)
                + contarNosComDoisFilhos(raiz->dir);
}

```

```

12. int raizPerfeitamenteBalanceada(NoArv* raiz) {
    if (raiz == NULL) return 0;
    return fatorBalanceamento(raiz) == 0;
}

```

```

13. int mostraCaminhoAVL(NoArv* raiz, int valor) {
    if (raiz == NULL) return 0;
    printf("%d ", raiz->valor);
    if (valor == raiz->valor) return 1;
    if (valor < raiz->valor) {
        if (mostraCaminhoAVL(raiz->esq, valor)) return 1;
    } else {
        if (mostraCaminhoAVL(raiz->dir, valor)) return 1;
    }
    return 0;
}

```

14. O caso descrito é **esquerda-direita (LR)**. Se o programador usar apenas `rotacaoDireita(raiz)`, a árvore continuará com a estrutura incorreta para esse tipo de desbalanceamento; em geral, o nó ainda permanecerá desbalanceado ou a reorganização ficará errada. O correto é aplicar `rotacaoEsquerdaDireita(raiz)`.

15. Erro: falta o 1 + no cálculo da altura.

Linha correta:

```
raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
```

16. Porque a convenção usada nas rotações e no restante do código é $fb = altura(esq) - altura(dir)$. Inverter para $altura(dir) - altura(esq)$ troca o sinal do fator de balanceamento e faz o algoritmo escolher rotações erradas (por exemplo, decidir uma rotação à esquerda quando o caso correto é à direita).

17. Falta tratar o caso em que o nó removido possui **dois filhos**. Nessa situação, é preciso encontrar o sucessor em ordem (menor valor da subárvore direita), copiar esse valor para o nó atual e depois remover recursivamente o sucessor. Sem isso, a função não remove corretamente nós com dois filhos.

18. (a) raiz final: 48;

(b) pai de 58: 55;

(c) o nó 37 fica como filho **direito** de 26;

(d) ocorreram **rotações simples**.

19. V, V, V, V.

```
20. int contarFolhasAVL(NoArv* raiz) {
    if (raiz == NULL) return 0;
    if (raiz->esq == NULL && raiz->dir == NULL) return 1;
    return contarFolhasAVL(raiz->esq) + contarFolhasAVL(raiz->dir);
}
```

```
int menorAVL(NoArv* raiz) {
    while (raiz->esq != NULL) raiz = raiz->esq;
    return raiz->valor;
}
```

```
int maiorAVL(NoArv* raiz) {
    while (raiz->dir != NULL) raiz = raiz->dir;
    return raiz->valor;
}
```

```
21. int avlIguais(NoArv* a1, NoArv* a2) {
    if (a1 == NULL && a2 == NULL) return 1;
    if (a1 == NULL || a2 == NULL) return 0;
    return (a1->valor == a2->valor) &&
        avlIguais(a1->esq, a2->esq) &&
        avlIguais(a1->dir, a2->dir);
}
```

22. (a) irmão de 40: 60;

- (b) folhas: 10, 25, 40, 60;
- (c) altura da árvore: 2;
- (d) ao inserir 70, a árvore permanece balanceada; **não há rotação**.

23. (a) menor valor: 5;
- (b) maior valor: 60;
- (c) fator de balanceamento do nó 20: 1;
- (d) ao inserir 2, ocorre **rotação simples à direita** na subárvore enraizada em 10.
24. (a) travessia em ordem: 20, 30, 40, 50, 60, 70, 80;
- (b) sucessor em ordem de 60: 70;
- (c) predecessor em ordem de 50: 40;
- (d) ao inserir 65, ele fica como filho **direito** de 60.
25. (a) ao remover 20, **não** é necessária rotação;
- (b) após remover 40, a subárvore esquerda da raiz fica com altura 0;
- (c) sim, a raiz continua balanceada (fator de balanceamento -1).
26. (a) nível do nó 20: 2;
- (b) nós internos: 4;
- (c) nós folha: 4;
- (d) após inserir 45, o fator de balanceamento do nó 50 passa a ser 0.
27. (a) nenhum nó fica desbalanceado;
- (b) **nenhuma** rotação é necessária;
- (c) inserindo 95 no lugar de 15, também **não** há rotação;
- (d) caminho de busca para 65: 55 75 65.
28. Erro lógico: a rotação dupla **direita-esquerda (RL)** foi implementada com as rotações trocadas. O correto é primeiro rotacionar **à direita** o filho direito e depois rotacionar **à esquerda** a raiz.
- ```
NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {
 raiz->dir = rotacaoDireita(raiz->dir);
 return rotacaoEsquerda(raiz);
}
```
29. Porque essa função apenas insere o novo nó como em uma ABB comum. Ela **não atualiza alturas, não calcula fator de balanceamento e não aplica rotações**. Portanto, sozinha, não garante que a árvore permaneça AVL após a inserção.
30. (a)

